

AURORA

Autonomous Upstream Reservoir Optimization and Risk-Aware Agent

Project Proposal

Keeping an AI controller for an oil reservoir within its limits
when the model it relies on is wrong

Course	SYSC 4907 — Engineering Project, Department of Systems and Computer Engineering, Carleton University
Supervisors	Prof. Safaa Bedawi and Prof. Rami Sabouni
Submitted	October 2026
Repository	https://github.com/ismaelouadria/AURORA

Name	Student no.	Program	Main responsibility
Ismael Ouadria	101284947	Software Engineering	Fast model, safety check, safety margin, overall design
Abdul Aziz Al Sibakhi	101264056	Software Engineering	Detailed reservoir simulation (OPM Flow + Egg)
Abdurrahman Al Choughri	[101xxxxxx]	Software Engineering	AI controller (reinforcement learning)
Mahdi Bouakline	101257788	Software Engineering	Experiments, logging, statistics, dashboard, and result interpretation
Timur Grigoryev	101276841	Computer Systems Engineering	Fault testing and failure handling

Summary. An AI controller decides how much water to pump into a simulated oil reservoir. Before each decision is applied, a safety check uses a fast, simplified model to predict whether it would push a well's pressure past its limit. If the model is wrong, a decision that looks safe can still break the limit. AURORA measures how wrong the fast model is by comparing it with a detailed simulator, turns that error into a safety margin, and then tests, on reservoirs the system has never seen, whether the margin reduces limit violations without giving up much oil production.

1 Introduction

Oil is held in the tiny pores of underground rock. As oil is pumped out, the pressure in the reservoir drops and production slows. *Waterflooding* is a common way to keep production going: water is pumped in through *injector* wells, which keeps the pressure up and pushes oil towards *producer* wells. Someone has to decide, every few weeks or months, how much water each injector should get (Figure 1). That is a control problem.

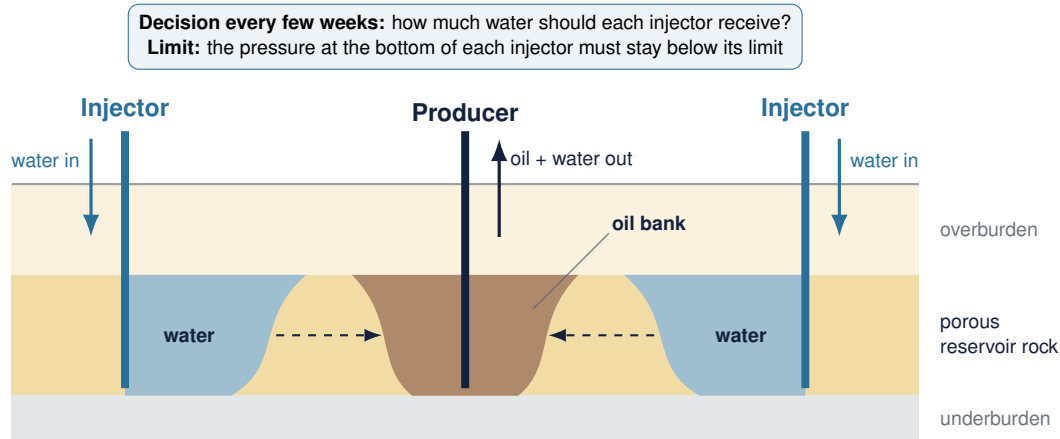


Figure 1: How a waterflood works. Injectors push water through the reservoir towards a producer. The decision to make is how much water each injector pumps over time.

Three things make it hard:

- **Limited view.** Nobody can see what is happening underground. The only measurements come from the wells: flow rates, the fraction of produced liquid that is water (the *water cut*), and the pressure at the bottom of each well (*bottom-hole pressure*, BHP).
- **Delayed effects.** Pumping more water today can raise production now but cause water to reach the producers sooner, which lowers the total oil recovered.
- **Slow or inaccurate models.** A detailed simulator is accurate but too slow to run for every decision being considered. A fast simplified model is quick but wrong by an unknown amount.

Reinforcement learning (RL), where a program learns good decisions by trial and error in a simulator, has been used for waterflooding several times [1, 2, 3]. Giving the controller a memory of past measurements is the usual way to deal with the limited view [4, 5]. However, an RL controller only tries to maximize its score. Nothing guarantees that each decision respects operating limits [6]. The usual fix is a *safety filter*: a check placed between the controller and the reservoir. It predicts the effect of each proposed decision and, if a limit would be broken, replaces the decision with the closest safe one [7].

The problem this project addresses. A safety filter can only catch what its model predicts. Suppose the pressure limit for an injector is 420 bar and the fast model predicts 414 bar for a proposed decision. The filter lets it through. If the detailed simulator then gives 423 bar, the limit is broken even though the filter worked as designed. The fast model was wrong, and nothing in the system allowed for that.

AURORA treats the error of the fast model as part of the safety problem. It measures that error

against the detailed simulator, focusing on the cases where the fast model predicts too low, and adds a safety margin of a size the measurements justify.

Scope. This is a software and computer engineering project that uses an oil reservoir as its application. The work is software design, connecting to an existing simulator, machine learning, optimization, statistics, testing, failure handling and visualization. Everything is tested in simulation, using a published test reservoir (the Egg model [8]) in an open-source simulator (OPM Flow [9]). We do not claim the system is ready for a real oil field, nor that our system surpasses those currently used in practice.

2 Objectives and Requirements

2.1 Objectives

Main objective. Build and test a software system that (1) controls water injection in a simulated reservoir using an RL controller with memory, (2) protects a pressure limit with a safety filter based on a fast model, and (3) finds out whether adding a margin based on the fast model’s measured error reduces limit violations on reservoirs the system has not seen before.

Each supporting objective has one owner, so it is clear who is responsible for it (Table 1).

Table 1: Supporting objectives, owners and how we show each one is done.

ID	Objective	Owner	Shown by
O1	Detailed simulation that our software can drive step by step: any Egg reservoir version running in OPM Flow	Aziz	Runs from a fresh copy of the repository with one script; tests on the extracted data
O2	AI controller that sees only well measurements, in two forms: without memory (PPO) and with memory (recurrent PPO)	Abdur-rahman	Saved settings and random seeds; a test proving no hidden data reaches the controller
O3	Fast model fitted to each reservoir version, and a safety filter that changes unsafe decisions as little as possible	Ismael	Report comparing the fast model with OPM; unit tests for the filter
O4	Measurement of the fast model’s error and a safety margin built from it	Ismael, Aziz	Dataset of paired fast-model and OPM results; report on how well the margin covers the error
O5	Fault testing and a supervisor that falls back to a safe action when something fails	Timur	Library of fault scenarios; tests of the supervisor
O6	Logging, an experiment runner, statistics and a dashboard	Mahdi	A result figure regenerated from its saved run record
O7	Test of the complete system on unseen reservoir versions	Team	Test plan fixed in advance; results with confidence intervals

2.2 Functional requirements

Every requirement is written so that it can be checked. Each has a priority (**Must**, **Should**, **Could**), an owner and a way of checking it (T = automated test, E = experiment, D = demonstration, I = inspection). Values marked *proposed* are fixed at the design review and changed afterwards only through a recorded decision.

Table 2: Functional requirements.

ID	The system shall. . .	Pr.	Owner	Ver.
Reservoir simulation				
FR-01	run any chosen Egg reservoir version in OPM Flow from settings kept under version control.	M	Aziz	T
FR-02	move the simulation forward one decision step at a time, applying injection rates supplied by our software.	M	Aziz	T
FR-03	after each step, read out well rates, total volumes, injector and producer BHP, and the pressure and water content of each cell, and save them in a fixed format.	M	Aziz	T
FR-04	check that the rates OPM applied equal the rates requested, and flag any difference.	M	Aziz	T
FR-05	produce matching fast-model and OPM results (same situation, same decision) for measuring the fast model's error.	M	Aziz, Ismael	T, E
AI controller				
FR-06	provide the simulation through the Gymnasium interface, with documented observations and actions.	M	Abdur-rahman	T
FR-07	give the controller only well measurements it is allowed to see; an automated check rejects any hidden simulator data.	M	Abdur-rahman	T
FR-08	train and test controllers with and without memory from settings files, with the random seeds recorded.	M	Abdur-rahman	E
FR-09	provide a fixed-rate controller and a simple rule-based controller as baselines, connected in the same way as the learned controllers.	M	Abdur-rahman	T
Fast model and safety filter				
FR-10	fit the fast model (CRM) to the injection and production data of a given reservoir version, and predict the response to a candidate decision one step ahead.	M	Ismael	T, E
FR-11	solve the safety filter's optimization problem (Equation (2)) for every proposed decision and output the applied decision, the size of the change and which limit caused it.	M	Ismael	T
FR-12	detect when no safe decision exists and apply the documented fallback.	M	Ismael, Timur	T
FR-13	compute the fast model's errors and the safety margin from the margin (calibration) reservoirs only.	M	Ismael	T, E
FR-14	run the safety filter with no margin, a fixed margin or the measured margin, chosen in the settings.	M	Ismael	T
Failure handling				
FR-15	create repeatable faults on demand: missing, frozen, delayed, noisy or invalid measurements; a drifting fast model; controller or solver faults; a failed simulator step.	M	Timur	T
FR-16	check each measurement (format, range, being a real number, how recent it is) before it reaches the controller.	M	Timur	T
FR-17	detect a part that does not answer in time, and switch to a degraded or fallback mode with a limited, logged response.	M	Timur	T
FR-18	record every fault, detection and change of mode in the logs.	M	Timur, Mahdi	T

ID	The system shall. . .	Pr.	Owner	Ver.
Logging, experiments and presentation				
FR-19	log, for each decision step, the observation, proposed decision, applied decision, fast-model prediction, margin, result and supervisor mode, with each stage kept separate.	M	Mahdi	T
FR-20	save a record of every run (code version, settings, reservoir version, seeds, simulator version, fingerprints of the input files).	M	Mahdi	T
FR-21	run every combination of controller, reservoir version and seed in a batch, and keep failed runs together with the reason they failed.	M	Mahdi	T, E
FR-22	compute the final-test measurements, with confidence intervals, from the logged runs.	M	Mahdi	T, E
FR-23	replay any logged run in a dashboard and compare controllers visually.	M	Mahdi	D
FR-24	let a presenter trigger a fault scenario in the dashboard and show the supervisor's response.	S	Timur, Mahdi	D
FR-25	fit the CRM to the Volve production data, tested on the last part of each record, as a check against a real field.	S	Ismael	E
FR-26	support, for comparison, an RL controller trained to respect limits by itself.	C	Abdur-rahman	E

2.3 Non-functional requirements

Table 3: Non-functional requirements with measurable targets.

ID	Quality	Requirement and target	Ver.
NFR-01	Reproducibility	A new team member can get a verified OPM run from a fresh copy of the repository using only the written steps; the simulator runs in a container with a fixed version.	D
NFR-02	Reproducibility	Parts with no randomness give identical decisions for identical inputs; results of trained controllers repeat within the variation reported between seeds.	T
NFR-03	Speed	<i>Proposed:</i> controller, fast model and filter together take under 1 s per decision on a team laptop; the filter alone has a median time under 50 ms.	T
NFR-04	Scaling up	The batch runner uses all available processor cores without changing the results.	T
NFR-05	Testing	Unit tests cover at least 80% of the lines in the safety, margin, supervisor and logging packages.	T
NFR-06	Fair testing	The reservoir versions kept for the final test are fixed and recorded in advance; no training, margin or tuning job may read them, and any attempt is logged.	T, I
NFR-07	Logging	Every decision step produces a complete log record; a missing field fails the run.	T
NFR-08	Failure handling	Under every fault we test, no invalid or out-of-range decision reaches the simulator.	T
NFR-09	Maintainability	Every package has a documented interface; every important design choice has a short written decision record.	I
NFR-10	Ease of use	A second reader can understand the dashboard’s main view without reading code, checked in a rehearsal with someone outside the team.	D
NFR-11	Portability	Runs on Linux and macOS with Docker; automated tests run on GitHub Actions.	T
NFR-12	Licensing	Equinor’s licence terms and credit for the Volve data are respected.	I

2.4 Use cases

Figure 2 shows who uses the system and for what. The users are the *experimenter* (any team member running the system) and the *reviewer* (supervisors, second reader or poster-fair visitor). The OPM Flow simulator is an external system. The three use cases at the heart of the project are UC1 (run a full simulation with the controller in the loop), UC4 (compute the safety margin from the matching runs) and UC6 (trigger a fault and observe the response). Their step-by-step descriptions are kept in the repository.

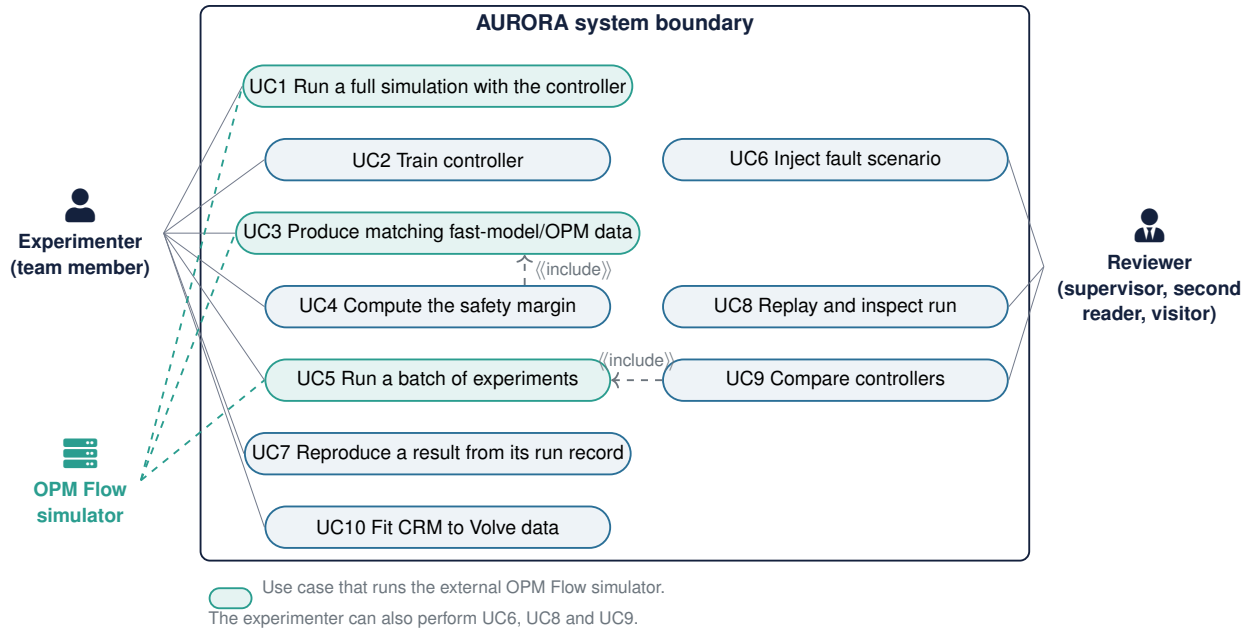


Figure 2: Use-case diagram. Solid lines join each user to their main use cases; dashed teal lines mark use cases that run the external OPM Flow simulator.

3 Background and State of the Art

Reservoir simulation and the Egg model. A detailed (“full-physics”) reservoir simulator divides the reservoir into thousands of small cells and solves the flow equations for oil and water in each one. OPM Flow is an open-source simulator of this kind that reads the standard input files used in industry [9]. The Egg model [8, 10] is a small made-up reservoir published for research on waterflooding. It has eight injectors and four producers, about 18,500 cells, and a simulated life of 3,600 days. It comes in 100 versions that differ in how easily fluid flows through different parts of the rock, which represents not knowing the true geology. Earlier work optimized injection on such models with gradient-based methods [11]. These are accurate but slow, and must be redone whenever the reservoir model changes.

Fast models (CRM). The capacitance–resistance model (CRM) describes a reservoir as a set of connections between injectors and producers. Each connection responds to a change in injection gradually, like a resistor–capacitor circuit [12, 13]. Its settings are fitted from injection and production data only, and it runs in microseconds where a detailed simulation takes minutes. CRM is widely used for forecasting [14]. It is known to be less accurate when the rock varies a lot from place to place [15], and Egg is built to vary in exactly that way. We therefore expect the fast model to be wrong, and the project is built around that.

Reinforcement learning with a limited view. When a controller cannot see the full state of the system, the best decision depends on what it has seen in the past [4]. Proximal Policy Optimization (PPO) [16] is a widely used RL algorithm. *Recurrent* PPO adds a memory unit (an LSTM [17]) so that the controller can remember earlier measurements. Both are available in the open-source library Stable-Baselines3 [18], which uses the standard Gymnasium interface between a controller and a simulation [19].

Safety filters. A safety filter changes unsafe decisions before they are applied [7, 20]. When

the predicted effect of a decision is a straight-line function of the decision, finding the closest safe decision is a small optimization problem called a quadratic program (QP). Solvers such as OSQP handle it in milliseconds [21]. Other approaches teach the controller the limits during training instead [22]; one was recently applied to waterflooding [23].

Measuring how wrong a model is. *Conformal prediction* is a simple statistical method for putting a bound on a model’s error. The model’s errors are measured on data set aside for that purpose, and a chosen percentile of those errors (for example the 95th) is used as the bound. If new cases are similar to the ones measured, the bound holds about that often [24, 25]. A one-sided version looks only at errors in the dangerous direction. Versions that adapt over time exist for data that change [26].

What is missing. We searched the literature specifically for work that would make this project unnecessary. Each piece exists already: RL for waterflooding, RL with safety limits for waterflooding [23], CRM and studies of its error [15], safety filters, and conformal prediction. In what we reviewed, we did not find them put together the way AURORA does: measuring the fast model’s error against a detailed simulator, turning the dangerous part of that error into a margin inside a safety filter, and testing the result on reservoir versions never used during development. Our contribution is that combination and a fair experiment that shows whether it helps. Any potential similar projects may exist internally for company use, but none are released or open source.

4 Proposed Solution and Methods

4.1 System design

AURORA has a *control loop* that runs once for every decision, and *preparation steps* done beforehand: training the controller, fitting the fast model and computing the margin (Figure 3). One pass through the loop works like this:

1. The simulator (OPM running an Egg reservoir) pauses at step k and reports the well measurements.
2. The supervisor checks that the measurements are valid and passes them to the controller as its observation o_k .
3. The controller uses o_k and its memory to propose injection rates u_k^{RL} .
4. The safety filter uses the fast model to predict the resulting pressure, adds the margin, and if needed changes the rates to the closest safe ones, u_k^* .
5. The simulator applies u_k^* , moves forward one step, and confirms it applied what was asked.
6. Every stage is logged separately: what was proposed, what the filter changed, what was applied and what happened.

The controller’s limited view is enforced in the code, not just by agreement. The underground pressures and the detailed simulator’s result go only to logging, margin calculation and final testing. An automated test fails if any of that data shows up in what the controller sees. Everything is written in Python and runs on team laptops or one workstation. OPM Flow runs in a Docker container with a fixed version, so every member uses exactly the same simulator. The parts exchange data in agreed formats, so members can build and test their own part before the others are finished.

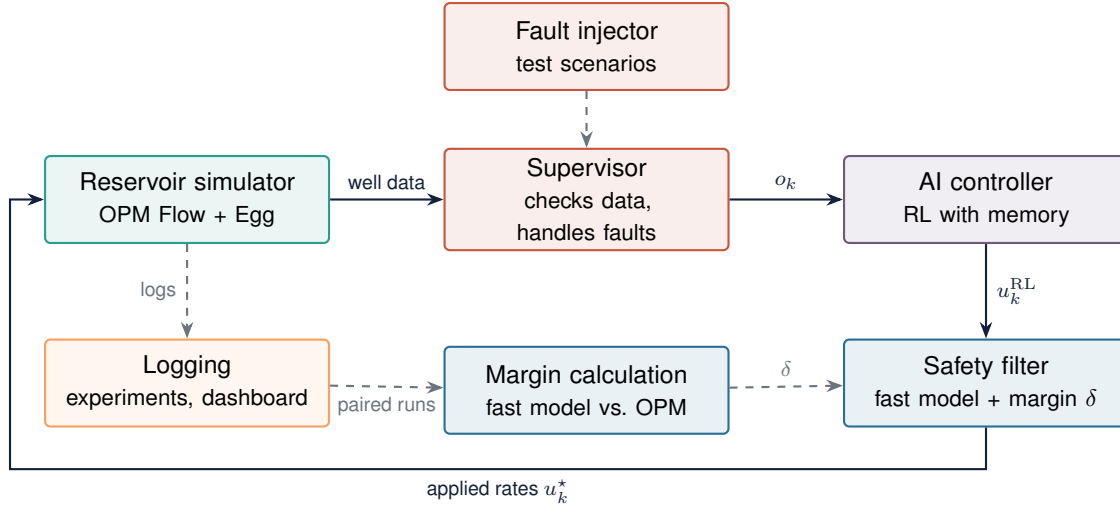


Figure 3: The AURORA control loop (solid arrows) and the supporting parts (dashed arrows). Colours show who owns each part: Aziz (teal), Timur (red), Abdurrahman (purple), Ismael (blue), Mahdi (orange).

4.2 Reservoir simulation (Aziz)

The Egg input files are prepared by a script from the published download and are never edited by hand. A reservoir version is chosen by swapping in its rock-property file. The controller has to change the injection rates at every decision and then continue the same simulation. Our default way to do this is to run OPM up to step k , save its state, and restart from that state with the new rates for step $k + 1$. OPM also has a Python interface that may be faster. Timing both options is the first implementation task. Every step is checked: the simulator finished without error, the expected output exists, the requested rates were applied, and the results make physical sense (no negative flows, water cut between 0 and 1, totals that add up). To measure the fast model’s error, the same saved state can be re-run with different candidate decisions, which gives the detailed simulator’s answer for exactly the situation the fast model was asked about.

4.3 AI controller (Abdurrahman)

The simulation is wrapped in the standard Gymnasium interface. Table 4 summarizes what the controller sees and does. It is deliberately not given the underground pressures, the rock properties, or which reservoir version it is running on. That is what gives it only a limited view.

Training directly in OPM may be too slow, because RL needs many simulated lifetimes. So most training is done in a fast stand-in simulation built from the fast model, and the controller is then refined and always tested in OPM. The versions with and without memory are trained under the same conditions, each with at least five different random seeds [27]. Two simple controllers serve as baselines: the fixed injection rates that come with Egg, and a hand-written rule that sends less water to injectors whose nearby producers are producing more and more water.

Table 4: Proposed observations, actions and score for the AI controller.

Observation	For each producer: oil rate, water rate, water cut, BHP. For each injector: the rate actually applied, BHP. Overall: fraction of time elapsed, the previous decision, and, from the supervisor, whether each value is valid and how old it is.
Action	8 injection rates, one per injector, each within the range the Egg model allows (up to 79.5 m ³ /d).
Decision interval	<i>Proposed:</i> every 90 days, which gives 40 decisions over the reservoir’s 3,600-day life. Chosen from 30, 60 and 90 days; shorter intervals give more control but cost more computing.
Score (reward)	Money earned in each step: oil revenue minus the cost of handling produced water and of injecting water. The prices are written down as an explicit assumption, and we test how sensitive the results are to them.
One run	One full reservoir life on one reservoir version from the development set.

4.4 Fast model and safety filter (Ismael)

In the form of CRM we use, each injector–producer pair has its own connection. The liquid that injector i contributes to producer j over a time step Δt is

$$q_{ij}(t_k) = q_{ij}(t_{k-1}) e^{-\Delta t/\tau_{ij}} + (1 - e^{-\Delta t/\tau_{ij}}) f_{ij} I_i(t_k), \quad (1)$$

where I_i is the injection rate, f_{ij} (between 0 and 1) is the share of injector i ’s water that reaches producer j , and τ_{ij} sets how quickly the producer responds. These values are fitted to OPM runs in which the injection rates are varied. The data used for fitting are never reused to judge how good the fit is.

The limit we protect is the bottom-hole pressure of each injector, which must stay below $y_{\max} = 420$ bar. This is the limit set in the Egg model. We use it as the rule for our experiment; we do not claim it is the pressure at which real rock would crack. CRM predicts flow rates, not pressure, so we add a simple straight-line relation between each injector’s rate and its pressure, fitted to OPM data and updated at each step with the latest measured pressure. The predicted pressure is then a straight-line function of the decision u : $\hat{y}_{k+1}(u) = c_k + G_k u$. Finding the closest safe decision becomes a small optimization problem:

$$u_k^* = \arg \min_{u, s \geq 0} (u - u_k^{\text{RL}})^{\top} W (u - u_k^{\text{RL}}) + \rho \mathbf{1}^{\top} s \quad \text{s.t.} \quad c_k + G_k u + \delta \leq y_{\max} + s, \quad u_{\min} \leq u \leq u_{\max}. \quad (2)$$

In words: stay as close as possible to what the controller proposed, while keeping the predicted pressure plus the margin δ under the limit, and the rates within their allowed range. The extra variable s lets the solver return an answer even when no fully safe decision exists. It is penalized heavily, and any use of it is logged as a failure of the filter. With $\delta = 0$ the filter trusts the fast model completely; we call this the *plain* filter. The problem is solved with OSQP [21].

Two details matter. First, the Egg input files tell the simulator itself to cap injector pressure at 420 bar by quietly pumping less water than requested. With that cap in place OPM would never report a pressure above the limit, and we could not see violations. For testing we raise the simulator’s cap, so that OPM reports the pressure the requested rate really needs. Second, if injector pressure never gets close to the limit, there is nothing for the filter to do and the comparison tells us nothing. We check this early. If it is the case, we switch to a limit on produced water, which the fast model

can also predict (Section 8).

Figure 4 illustrates the idea behind the margin, which the next subsection explains.

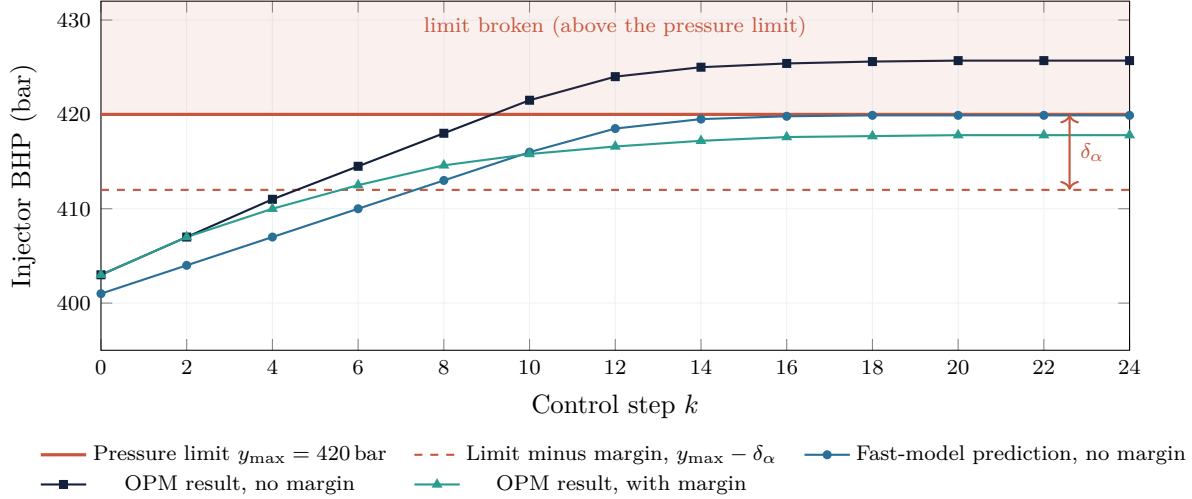


Figure 4: Illustration of the idea (not experimental data). A filter with no margin keeps the *predicted* pressure below the limit, but the fast model predicts too low, so the pressure in the detailed simulator goes above it. Adding the measured margin δ_α keeps the actual pressure below the limit while staying close to it.

4.5 Measuring the fast model’s error (Ismael with Aziz)

For the same situation and the same decision, let y_{k+1}^{OPM} be the pressure from the detailed simulator and $\hat{y}_{k+1}^{\text{ROM}}$ the fast model’s prediction. The error is

$$r_{k+1} = y_{k+1}^{\text{OPM}} - \hat{y}_{k+1}^{\text{ROM}}. \quad (3)$$

A positive error is the dangerous kind: the fast model predicted too low, so the filter thinks an unsafe decision is safe. We collect n such errors on the reservoir versions set aside for this, sort them from smallest to largest, and take the value below which a chosen share of them fall. With a target of 95% (that is, $\alpha = 0.05$):

$$\delta_\alpha = r_{(\lceil (n+1)(1-\alpha) \rceil)}. \quad (4)$$

This is the one-sided conformal bound described in the background [24]. In theory the fast model’s error on a new case is then below δ_α at least 95% of the time, provided the new cases resemble the measured ones. Reservoir data only partly meet that condition, because errors close together in time or at neighbouring wells are related. So we treat 95% as a target and report what is actually achieved. To estimate how well the margin carries over to a new reservoir version, we repeatedly leave one version out, compute the margin from the rest, and check it on the one left out. We also compute a separate margin for each injector. If the margin does not carry over, the fallback is a version that adjusts itself over time [26]. The planned amount of data is 20 reservoir versions $\times$ 20 points in time $\times$ 5 candidate decisions, about 2,000 short OPM runs.

4.6 Fault testing and failure handling (Timur)

A control system that has only been tested with perfect data has not really been tested. This part has two pieces: a *fault injector* that creates realistic problems on demand, and a *supervisor* that detects them while the system runs and keeps it in a defined safe mode. The fault scenarios are based on evidence, not invented: our review of real field data (??) shows missing pressure readings, wells that change role and impossible negative volumes. Table 5 lists the kinds of fault.

Table 5: Kinds of fault, what each is based on, and the expected response.

Kind of fault	Examples	Based on	Response
Missing data	sensor drops out; gaps that come and go	missing pressure in Volve	keep the last good value for a limited time; degraded mode
Frozen or delayed data	value stops changing; old measurement arrives late	common failures of measurement systems	check how recent the data are; degraded mode
Invalid data	not a number, negative rate, water cut above 1	Volve records with negative volumes	reject the value; degraded mode
Noise and spikes	random noise, outliers	measurement noise	smooth the input; log
Model drift	fast-model values shifted; unfamiliar reservoir	known CRM error [15]	larger margin, or fallback
Controller fault	invalid or out-of-range decision; no answer in time	software failure	reject; fallback
Filter fault	no safe decision found, or solver too slow	optimization failure	fallback
Simulator fault	OPM step fails or takes too long	numerical failure	mark the run invalid; keep it

The supervisor is a small state machine that always behaves the same way for the same inputs (Figure 5). It checks every measurement (right format, sensible range, a real number, recent enough) and uses timers to detect parts that stop responding. In *Degraded* mode the controller is given the last good values, as long as they are not too old, and the filter may use a larger margin. In *Fallback* mode a cautious action is applied: the last decision known to be safe, reduced by a documented factor. *Safe hold* ends the run and flags it. The supervisor waits for conditions to stay good for several steps before switching back, so it does not flip rapidly between modes.

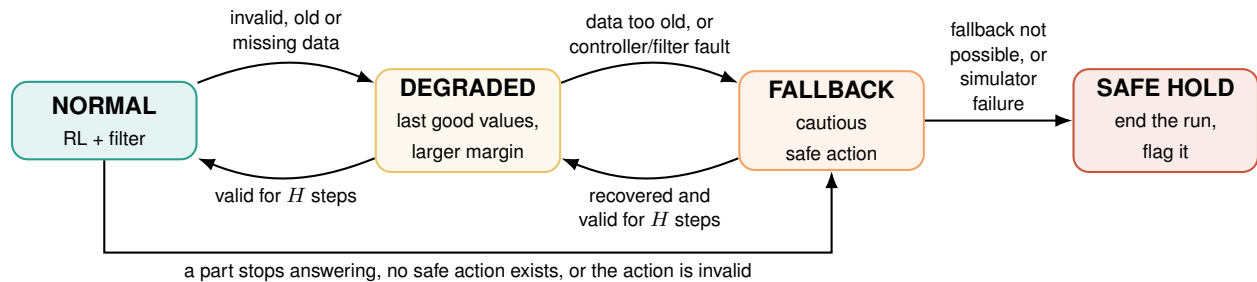


Figure 5: The supervisor's state machine. Every change of mode is logged with its cause, and the time to detect a fault is measured from the moment the fault injector started it.

Each scenario is an automated test with an expected sequence of modes. We report how often faults

are detected, how quickly, how many false alarms occur on clean runs, and how much performance is lost.

4.7 Logging, experiments and dashboard (Mahdi)

One log record is written per decision step, with each stage stored separately: the measurements, the proposed decision, the fast model’s prediction, the margin, the applied decision, what the simulator did, the result, the supervisor’s mode and timings. Every run also saves a record of exactly how it was produced: the version of the code, the settings, which reservoir version, the random seeds, the simulator version and whether it succeeded. A batch runner works through every combination of controller, reservoir version and seed in parallel. It can resume after an interruption, and it keeps failed runs, with the reason, instead of discarding them. The dashboard reads only these logs, so it shows what actually happened.

5 Evaluation: How We Will Know Whether It Worked

5.1 Research question and hypotheses

Research question. If the safety filter includes a margin based on the fast model’s measured error, does the controller break the pressure limit less often on reservoir versions it has never seen, compared with the same controller and a filter with no margin, and without losing much oil production?

- H1 *The error matters.* The fast model predicts too low often enough that the plain filter still lets violations through in OPM.
- H2 *Main hypothesis.* On unseen reservoir versions, the filter with the measured margin has fewer or smaller violations than the plain filter.
- H3 *Cost.* For a similar number of violations, the measured margin gives up less oil (or money) than a fixed, guessed margin.
- H4 *Memory.* Given the same measurements, the controller with memory does better than the one without.

5.2 Experiment design

This follows the usual machine-learning practice of separate training and test data, except that we split reservoir versions instead of rows of a table. The 100 Egg versions are divided once: 60 for development (training, tuning and demonstrations), 20 for measuring the fast model’s error, and 20 kept unseen for the final test. The assignment is random with a fixed seed and is saved in the repository. The experiment software refuses to load the test versions until the test plan has been fixed (Figure 6).

The main experiment compares seven controllers, each adding one feature to the one before (Table 6). The comparisons that answer the research question are C4 against C6 and C5 against C6.

Everything we report is measured from what happened in the detailed simulator, never from the fast model’s predictions (Table 7).

Test plan and statistics. Before the final test we fix everything that could affect the result and

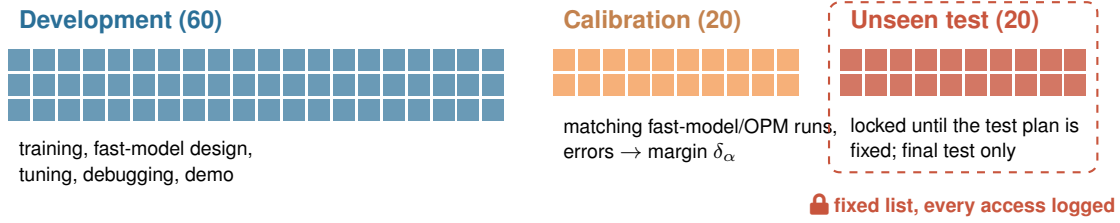


Figure 6: Proposed split of the 100 Egg reservoir versions. No version moves between groups once the split is fixed.

Table 6: The controllers compared on unseen reservoir versions.

ID	Controller	What it tells us	Hyp.
C0	Fixed rates that come with Egg	The baseline: no control at all	–
C1	Hand-written rule	Does learning beat a simple rule?	–
C2	RL without memory, no filter	How well learning does without memory	H4
C3	RL with memory, no filter	Whether memory helps	H4
C4	RL with memory + plain filter (no margin)	Does trusting the fast model still allow violations?	H1
C5	RL with memory + fixed margin (several sizes)	What a guessed margin costs in production	H3
C6	RL with memory + measured margin	Does a margin based on measured error help?	H2, H3

Table 7: What we measure.

Group	Measurements
Production and money	Total oil, total water produced and injected; the money-based score, and how much it changes with the assumed prices
Safety	How often the limit is broken; the largest and the total amount above the limit; how long violations last
Filter behaviour	How often the filter changes a decision and by how much; how often no safe decision exists
Fast-model accuracy	The spread of its errors; average error size; how bad the worst under-predictions are; by reservoir version and well
Margin	How often the margin actually covers the error, compared with the 95% target, on margin and on unseen reservoirs
Failure handling	How often and how quickly faults are detected; false alarms; performance lost under faults
Computing time	Time per decision for the controller, fast model and filter; OPM time per step

save it as a tagged version of the repository: the split of reservoir versions, what the controller sees and controls, the prices in its score, the 95% target, what we measure, the fixed margin sizes, the number of seeds and the statistical method. The 20 test versions are then used once. Each learned controller is trained with at least five random seeds, and each is tested on all 20 test versions. Because every controller is tested on the same versions, we compare them version by version. We report 95% confidence intervals computed by bootstrap resampling [28], and we only claim a difference when the interval does not include zero. The final test is about 840 full simulations. If one takes T minutes, the total is about $14T$ hours of computing, which is why measuring T comes first.

5.3 Testing

Each design level has a matching test level (the V-model), and every requirement in Section 2 states how it is checked. Table 8 lists the levels. Automated tests run on every proposed code change. They show that our written checks pass; whether the results make physical sense is covered in the next subsection.

Table 8: Test levels.

Level	Examples	When it runs
Unit and random-case	The filter changes decisions as little as possible and reports when no safe decision exists; the margin is the right percentile; CRM fitting recovers known values from generated data; supervisor mode changes	Every code change
Interface	Each data format exchanged between two parts is checked, using stand-ins for the part that produces it and the part that uses it	Every code change
Integration	Simulation wrapper with OPM for one step; controller with filter; logs are complete	Nightly or on request
System	A full simulation on a development reservoir reproduces a stored reference result	Before each milestone
Fault tests	Every fault scenario gives its expected sequence of modes and never an invalid decision	Before each milestone
Final test	All controllers compared on the unseen reservoirs under the fixed test plan	Once (M6)

5.4 Telling right results from wrong ones

No one on the team is a reservoir engineer, and we cannot ask an expert to review our system. So we check correctness at four levels:

- *Does each part work?* Every component is tested on a case where we already know the answer. If we generate data from a CRM with known settings, fitting must recover those settings. If a decision is already safe, the filter must return it unchanged. On made-up errors with a known distribution, the margin must cover 95% of them.
- *Did the experiment run as planned?* Saved run records and the guard on the test versions show that the experiment ran as specified. We also check results that can be predicted in advance: a margin of zero must behave exactly like the plain filter, and a bigger margin must never cause more violations.

- *Do the results make physical sense?* Basic rules are checked at every step, and we check that changes go in the right direction (pumping less water must not raise the pressure at that well). Our run of the original Egg settings is compared with the reference results published with Egg.

We write down what each possible result would mean before we see the results (Table 9). Finding that the margin does not help is an acceptable outcome, and we would report it along with the reason.

Table 9: What each possible result would mean, decided in advance.

If we see...	It means...
C6 has fewer violations than C4, and more oil than C5 for similar violations	H2 and H3 are supported, for the conditions we tested
C6 and C4 about the same, both with almost no violations	The limit is rarely reached, or the fast model is good enough; H1 is not supported; try the produced-water limit
C6 and C4 about the same, both with violations	The margin is too small, or it did not carry over to new reservoir versions
C6 is safe but no better than C5	A measured margin has no advantage over a guessed one here; H3 is rejected
C3 and C2 about the same	Memory does not help with these measurements; H4 is rejected

5.5 Demonstration

The supervisors were clear that a demonstration must not be numbers scrolling in a terminal. The system is shown through an interactive dashboard

Oral presentations are in the week of January 25–29. The course allows 20 minutes per member, including a demonstration, and each member presents their own part (Table 10). Our target for that date is the complete loop, including the measured margin, working from start to finish on development reservoirs. At the poster fair (March 19) the dashboard replays recorded runs next to the poster, whose centre is the final result on the unseen reservoirs. Demonstrations use only development reservoirs, so the test reservoirs stay unseen.

Table 10: Planned demonstration: who presents what.

Presenter	Part
Ismael	The problem and the system design; then the safety filter: replay of a step where the plain filter lets a violation through and the filter with the measured margin does not
Aziz	The reservoir: Egg in OPM, one step run live, and the test that shows we can change a decision and see the effect
Abdurrahman	The controller: what it sees, what it proposes, and training curves with and without memory
Timur	Failure handling: trigger a frozen-sensor fault and a filter-timeout fault live; watch the detection and the fallback
Mahdi	Evidence: comparison of the controllers on development reservoirs, tracing one result back to its run record, and what is left to do

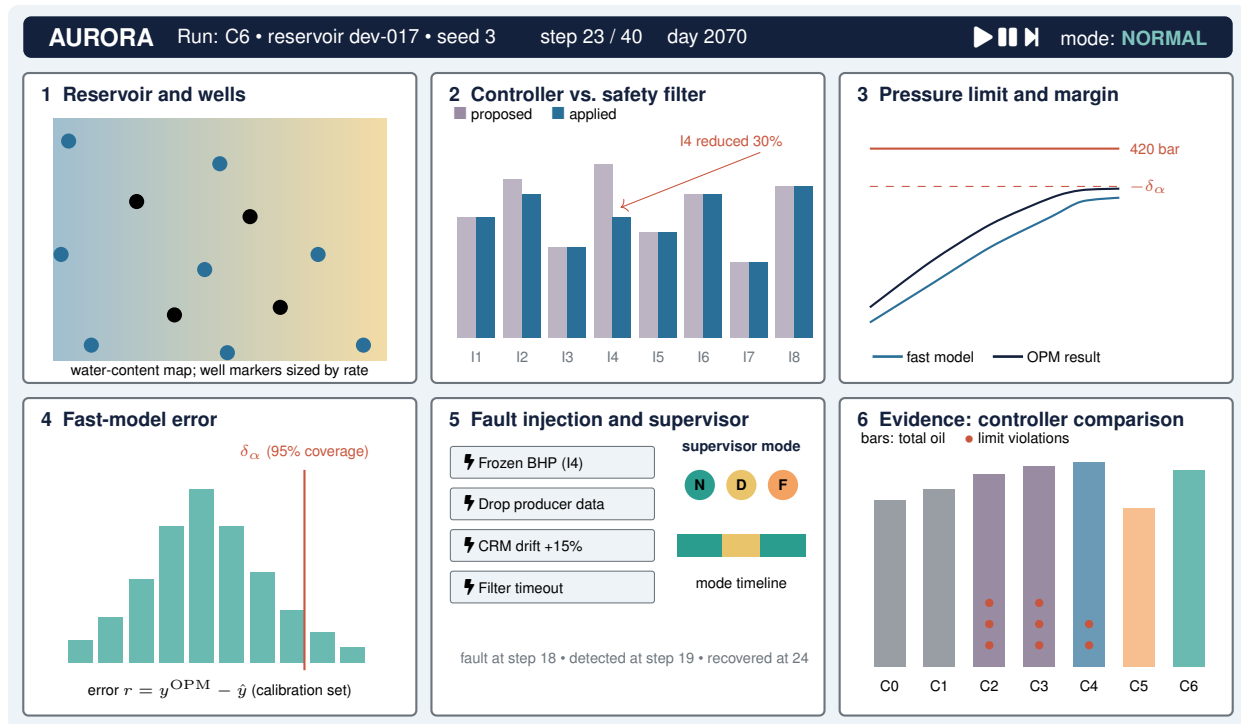


Figure 7: Sketch of the demonstration dashboard. Every panel is filled from the logs; a run can be played step by step or moved to any point.

6 Team, Degree Relevance and Skills

AURORA is a software and computer engineering project whose application happens to be an oil reservoir. The real work is designing, building and testing a control system that uses data and learning, includes optimization, keeps working when parts fail, and keeps a full record of its results. The project applies methods we have already studied; the reservoir is the setting, not the subject.

6.1 How the project relates to each member's degree program

Ismael Ouadria (Software Engineering). Ismael owns the fast model, the safety filter and the safety margin, and the overall software design. This is software engineering applied to optimization and statistics: each piece is built as a tested module with a defined interface, and the design of how the parts connect draws on software architecture and requirements courses.

Abdul Aziz Al Sibakhi (Software Engineering). Aziz owns the detailed reservoir simulation. His work is fitting a large existing scientific program (OPM Flow) into an automated, repeatable pipeline that runs in a container: reading and checking its output, running it in parallel, and providing the simulation that every claim is checked against. This is software integration, testing and automation.

Abdurrahman Al Choughri (Software Engineering). Abdurrahman owns the AI controller. His work is machine-learning engineering: designing the training setup, controlling what the model is allowed to see, training repeatably, and testing fairly across random seeds and unseen data. It builds on machine-learning and software-testing coursework.

Mahdi Bouakline (Software Engineering). Mahdi owns the experiment platform, the statistics

and the dashboard. His work is data and experiment infrastructure: log formats, a record of every run, a batch runner, automated tests, statistical analysis and a user-facing visualization application. It draws on software design, software testing and statistics courses. These skills we are all obtained through the courses he has taken in his program, and his coop work experience.

Timur Grigoryev (Computer Systems Engineering). Timur owns fault testing and failure handling. His work is dependable real-time software: defining faults, creating them on purpose, monitoring the running system, detecting parts that stop responding, and falling back safely, all checked by automated tests. It draws on real-time and concurrent systems courses and matches his software background and internship experience.

6.2 Ownership and deliverables

Capstone grading is individual, so every member owns one part of the system from design through to demonstration (Table 11). Each part also has a named backup who reviews every change to it and could carry on the work. The backups form a ring, so every member reviews one other part and is reviewed by someone else.

Who wrote what. [To be filled in by the team before submission: which member wrote each section of this proposal.]

Table 11: Ownership, backups and what each member delivers.

Member	Owns	Backup	Deliverables that can be assessed individually
Ismael	Fast model, safety filter, safety margin, overall design	Aziz	Fast-model code and a report comparing it with OPM; safety filter (3 modes); saved margins and coverage report; CRM study on Volve data; design decision records
Aziz	OPM Flow + Egg simulation; matching runs; runs on unseen reservoirs	Ismael	Scripts that set up Egg and OPM; simulation wrapper with stepping and branching; code to read OPM output; dataset of matching runs; OPM runs for the final test
Abdur-rahman	RL training setup; baseline controllers; RL with and without memory	Mahdi	Gymnasium setup; test that no hidden data reaches the controller; trained controllers; report comparing the controllers (H4)
Mahdi	Logging, run records, experiment runner, statistics, dashboard	Timur	Log format; run records; batch runner; statistics code; dashboard; analysis of the final test
Timur	Fault injector; supervisor; fault tests	Abdur-rahman	Library of fault scenarios; supervisor; automated fault tests; report on the fault tests; live fault panel

6.3 Skills of the team as a whole

The five roles were chosen so that the team's skills cover every part of the system (Table 12). Together we cover the three areas the project combines: software engineering (design, testing, real-time monitoring), data-driven methods (learning, statistics) and numerical modelling (simulation, fast models, optimization). The one thing none of us is trained in is reservoir engineering. Section 5 explains how the project is set up so that its conclusions do not depend on having an expert check every run.

Table 12: Team skills matched to project needs. ● = leads this area, ○ = supports or is the backup.

Skill the project needs	Ismael	Aziz	Abdurrahman	Mahdi	Timur
Software architecture and interface design	●	○			
Working with existing scientific software (OPM Flow, containers)	○	●			
Reinforcement learning and ML engineering			●	○	
Optimization and fast simplified models	●	○			
Statistics and experiment design	●			○	
Data pipelines, record-keeping and reproducibility				●	○
Visualization and user-facing application				○	●
Fault testing, monitoring a running system, safe fallback			○		●
Software testing, automated test runs and code review	○	○	○	○	○

6.4 Methods and the courses they build on

The project is built with established engineering methods. Table 13 lists each method, where we use it, and the coursework it builds on.

Table 13: Methods used and the coursework they build on. Course codes are to be confirmed by each member against their own program.

Method	Where we use it	Coursework
Breaking a system into parts with clear interfaces	Code structure, agreed data formats, recorded design decisions	SYSC 3110, SYSC 4120
Requirements analysis	Checkable functional and non-functional requirements, use cases	SYSC 3120
Software testing (V-model)	Unit, interface, integration and system tests; automated test runs	SYSC 4101
State machines and timing checks	Supervisor, detecting parts that stop responding, parallel simulation runs	SYSC 3303, SYSC 4001
Machine learning with separate test data	RL controllers, performance on unseen reservoirs, keeping hidden data from the controller	SYSC 4415
Modelling and optimization	Fitting the fast model, setting up the safety filter, analysing model error	SYSC 3600; optimization
Statistics	Margin from measured errors, confidence intervals, experiment design	STAT 3502
Data engineering	Data formats, run records, repeatable pipelines, problems in real data	Database and project courses
Planning around risks	Risk list, milestones, cost, being clear about limitations	Engineering profession courses; co-op

7 Project Management and Schedule

7.1 How we work

We use a light version of Agile: a Kanban board on GitHub organized by milestone, with each requirement linked to its test. A strict step-by-step (waterfall) plan has too little room for the unknowns of a research question, and full Scrum has too many meetings for a part-time team.

- **Tasks.** Each task states one outcome small enough to show progress within about a week, what must be true for it to count as done, an owner and a reviewer.
- **When a task is done.** Its conditions are met, tests pass, results are saved, the backup has reviewed it, and it is merged into the main branch. A milestone is done when its conditions are met, not when its date passes.
- **Handling changes.** Changes to scope, data formats, limits or the test plan are written down as decisions, not made quietly. The progress report will explain any difference from this proposal.
- **Individual contribution.** Each member's contribution is shown by records that last: tasks closed, code changes written, reviews done as backup, experiment records and reports, and their part of the demonstration and reports.

7.2 Expectations agreed with the supervisors

We presented the project to the supervisors and agreed the expectations in Table 14. Any change will be recorded in the meeting minutes.

Table 14: Expectations agreed with the supervisors and how the project meets them.

Expectation	How it is met	Where
Every part of the system is needed to answer the research question	Every part feeds the comparison of controllers; the fault tests run on the same loop	Section 4
Fault scenarios are based on real data and published sources	The fault library comes from data problems found in Volve and from published failure types	Section 4 and ??
The team can judge whether results make physical sense without an expert on every run	Checks at four levels, automated physical checks, scheduled expert review	Section 5
The demonstration can be followed by a second reader from any background	Interactive dashboard; each member presents their own part	Section 5
Grading is individual: each member owns a documented part and reports on it every week	Named owners, a ring of backups and records of each member's contribution	Section 6
Weekly meetings on Thursdays 9:00–10:00, with minutes sent after each one	Members take turns writing minutes and send them the same day	This section

Every member reports on their own work at every weekly meeting, even when there has been little progress. Before each meeting each member updates their current task with the result, anything blocking them and the next step. The supervisors can view the repository and the project board, so they can see progress between meetings. At the end of each milestone the team compares what was delivered with the plan and records any delay and its cause.

7.3 Milestones

The work is divided into milestones of a month or less, each ending with something that can be shown (Table 15). Course dates are the tentative ones announced at the course information session.

Table 15: Milestone plan.

ID	Dates	What exists at the end
M1	Sep 28 – Oct 23	Foundations. <i>Proposal submitted (Oct 19–23)</i> ; requirements and data formats drafted; Egg running in OPM from a script; a complete loop with a fixed controller and a basic filter, fully logged
M2	Oct 26 – Nov 20	First version of each part. OPM running one decision at a time with pressure output; RL setup and a first trained controller; fast model fitted to Egg; plain safety filter; first fault injector; run records and batch runner
M3	Nov 23 – Dec 11	AI controller in the loop. RL with memory plus the plain filter running in OPM; first measurements of the fast model’s error (evidence for H1); first supervisor; dashboard views 1–3; <i>progress report (Dec 11)</i>
–	Dec 12 – Jan 4	Exams and break: spare time only
M4	Jan 5 – Jan 29	Measured margin. Matching runs on the margin reservoirs; margin and coverage report; filter with the measured margin working; live fault demo; dashboard views 1–5; <i>oral presentation and demonstration (Jan 25–29)</i>
M5	Feb 1 – Feb 20	Ready for the final test. All fault scenarios and a report on them; batch runner handling the full workload; fixed-margin comparison; <i>test plan fixed (Feb 20)</i>
M6	Feb 23 – Mar 19	Final test. All seven controllers on the unseen reservoir versions; statistics; <i>final-report draft to supervisors (Mar 8–12)</i> ; <i>poster fair (Mar 19)</i>
M7	Mar 22 – Apr 9	Wrap-up. <i>Final report and video (Apr 9)</i> ; check that results can be reproduced; final release

The work that everything else waits on is the reservoir simulation. Running OPM one decision at a time is needed before the controller can run in OPM, before the matching runs for the margin, and before the final test. It is therefore scheduled first. The other parts are built against simple stand-ins in the meantime, so everyone can work in parallel. Fixing the test plan on February 20 is a firm rule: no run on the unseen reservoirs happens before it. Figure 8 shows who works on what and when.

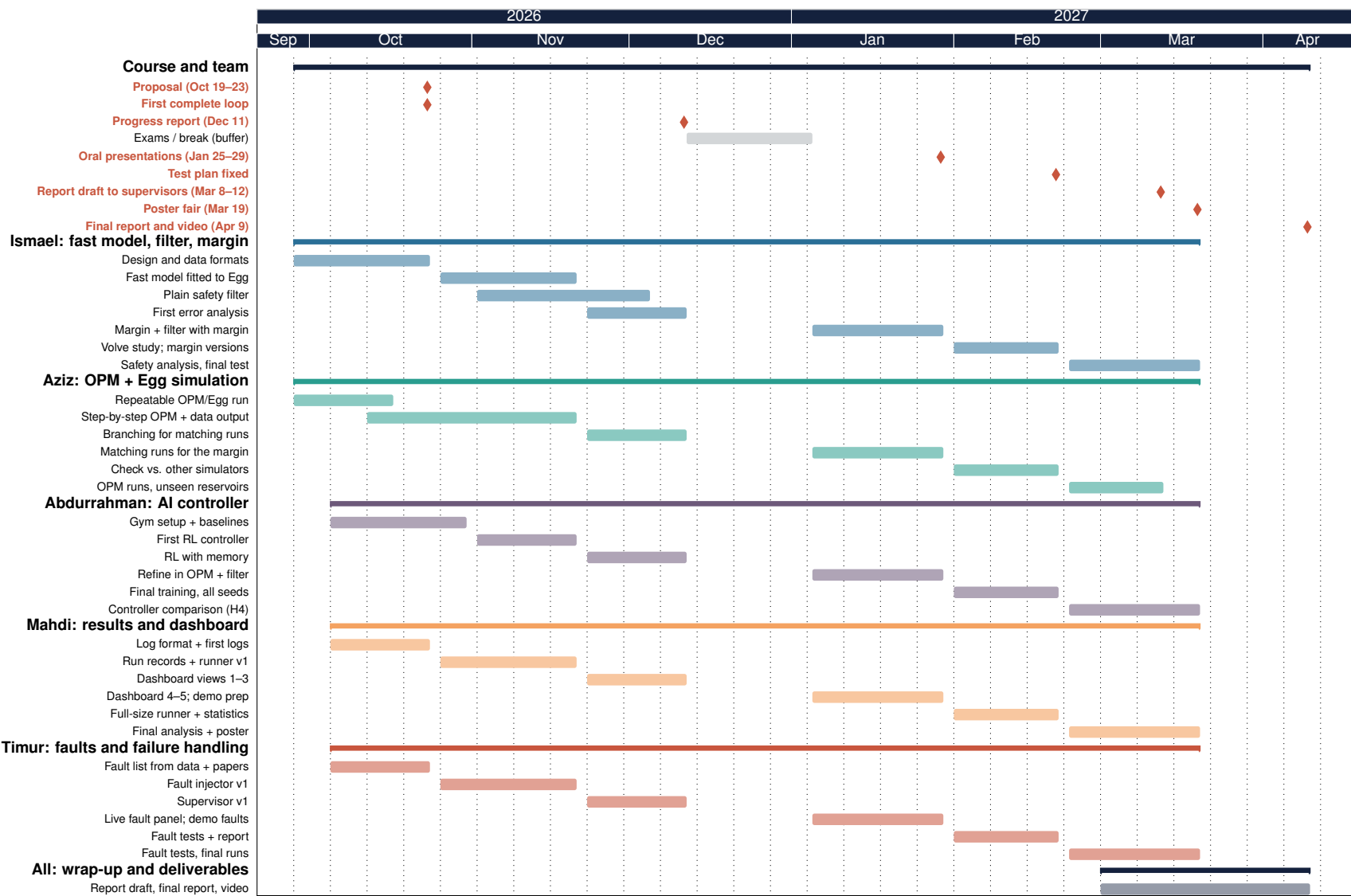


Figure 8: Time and person chart. Bars are coloured by owner; red diamonds are course and project milestones (course dates are the tentative ones from the SYSC 4907 information session).

8 Risks and Mitigation

Risks are kept in a list in the repository and reviewed every week. Each has an owner, an action we are taking now, and a condition that tells us to switch to a fallback planned in advance (Table 16). Likelihood (L) and impact (I) are rated Low, Medium or High.

Table 16: Risks, what we do about each, and when we switch to the fallback.

ID	Risk	L	I	What we do about it; when we switch → fallback	Owner
Subject knowledge and research					
R1	No one on the team is a reservoir engineer, so we may misjudge results	H	H	Automated physical checks; meaning of each result written in advance; scheduled expert review (Section 5). A result fails a physical check → throw the run out and investigate.	Ismael
R2	The pressure limit is rarely reached, so the safety comparison shows nothing	M	H	Measure early how close pressures get to the limit. Fewer than 5% of steps come near it → switch to the produced-water limit.	Ismael
R3	The fast model’s error is too large for a margin that is still useful	M	M	Try the per-pair CRM and versions for different operating conditions. The margin leaves the controller almost no room to act → report that the fast model is not good enough.	Ismael
R4	The margin does not carry over to new reservoirs	M	M	Leave-one-reservoir-out checks. Coverage well below 95% → per-well or self-adjusting margins; report the failure as a finding.	Ismael
R5	The controller with memory trains poorly	M	M	Start from a working controller without memory. No progress by M3 → use the simpler fallback and report H4 as unanswered.	Abdur-rahman
R6	Test reservoirs leak into development, making the result invalid	L	H	The split is fixed and saved; the software refuses to load test reservoirs before the test plan is fixed (NFR-06).	Mahdi
R7	Closer earlier work turns up	L	M	Keep watching the literature; word our claims as “in the work we reviewed”.	Ismael
Technical					
R8	Running OPM one decision at a time is too slow	M	H	Time the options first. One full simulation takes over 30 minutes → train on the fast model, make decisions less often, run in parallel.	Aziz
R9	The parts do not work together	M	M	Data formats between parts agreed early, with stand-ins and tests of each format; a minimal complete loop in M1.	Ismael
R10	Software versions change and results can no longer be reproduced	L	M	Fixed OPM container version; fixed versions of Python libraries; automated tests on every code change.	Mahdi
R11	Failure handling hides real errors, or flips between modes	L	M	The supervisor waits before switching back; every change of mode is logged; false alarms are measured on clean runs.	Timur

ID	Risk	L	I	What we do about it; when we switch → fallback	Owner
Project and team					
R12	Too much knowledge sits with one member	M	H	Owner and backup for every part; documentation in the repository; every member presents their own work weekly.	All
R13	Late start squeezes the fall schedule	H	M	Get a minimal loop working first; milestones of a month or less; optional comparisons can be dropped.	Ismael
R14	A member is unavailable (illness, heavy workload, exams)	M	M	The backup carries on; during exam periods no essential work depends on one person; spare time over the winter break.	All
R15	The project keeps growing	M	M	Any addition to the scope needs a recorded decision and a stated effect on the schedule.	Ismael
R16	The demonstration is not understandable to a second reader	L	H	Dashboard designed early; rehearsal with someone outside the team before the oral presentation (NFR-10).	Mahdi

9 Special Components, Facilities and Cost

The project needs no hardware to be bought or built (Table 17). It is well within the \$500 budget each group has. The only possible cost is renting cloud computers if our laptops and the department's machines are too slow for the final test. A cloud machine with 8–16 processor cores costs roughly \$0.40–\$0.80 per hour, so 200 hours would cost under \$150. No graphics card is needed: the controllers are small neural networks, and the simulator runs on ordinary processors.

Table 17: Components and facilities required.

Component or facility	Needed for	Availability; cost
OPM Flow simulator (in Docker)	Detailed simulation for error measurement and final test	Open source; in use; \$0
Egg model, 100 versions	Reservoirs for training, margin and final test	Public; downloaded; \$0
Volve field data	Realistic measurements and fault scenarios	Free licence; downloaded; \$0
Python libraries (Stable-Baselines3, CVXPY/OSQP, Streamlit)	Controller, safety filter, analysis, dashboard	Open source; \$0
GitHub repository, automated tests and project board	Version control, testing, planning	Free for students; \$0
Laptops	Development and small experiments	Members' own; \$0
Multi-core workstation or cloud computer	Running many simulations in parallel for the final test	Department access [to confirm]; \$0–\$150
Screen and space at the poster fair; poster printing	Live demonstration	Provided by the course [to confirm]; \$0–\$60
Total direct cost		\$0–\$210

10 Limitations and Expected Outcome

All of our comparisons are done in simulation, on a made-up reservoir. OPM is more accurate than our fast model, but it is still a simulator, and Egg is not a real field. The safety filter protects the limit we give it; 420 bar is Egg's limit, not the pressure at which real rock would crack. The 95% target for the margin rests on a statistical assumption that reservoir data only partly meet, so we report what is actually achieved. All 100 Egg versions are the same kind of reservoir, so our results may not carry over to other kinds.

By the end of the project we will deliver a working control system whose results can be reproduced; a comparison of seven controllers on unseen reservoir versions, with confidence intervals, showing which hypotheses hold and which do not; a dataset of the fast model's errors and a library of fault scenarios; and the course deliverables built from this work. The kind of claim we expect to make is: "the measured margin reduced limit violations on 20 unseen Egg reservoir versions in OPM Flow." Whether the main hypothesis turns out true or false, the answer will be backed by evidence.

References

- [1] F. Hourfar, H. J. Bidgoly, B. Moshiri, K. Salahshoor, and A. Elkamel, "A reinforcement learning approach for waterflooding optimization in petroleum reservoirs," *Engineering Applications of Artificial Intelligence*, vol. 77, pp. 98–116, 2019.
- [2] H. Ma, G. Yu, Y. She, and Y. Gu, "Waterflooding optimization under geological uncertainties by using deep reinforcement learning algorithms," in *SPE Annual Technical Conference and Exhibition*, no. SPE-196190-MS, 2019.
- [3] K. Zhang, Z. Wang, G. Chen, L. Zhang, Y. Yang, C. Yao, J. Wang, and J. Yao, "Training effective deep reinforcement learning agents for real-time life-cycle production optimization," *Journal of Petroleum Science and Engineering*, vol. 208, p. 109766, 2022.
- [4] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.
- [5] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPs," arXiv:1507.06527, 2015.
- [6] J. García and F. Fernández, "A comprehensive survey on safe reinforcement learning," *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015.
- [7] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, p. 109597, 2021.
- [8] J. D. Jansen, R. M. Fonseca, S. Kahrobaei, M. M. Siraj, G. M. Van Essen, and P. M. J. Van den Hof, "The Egg model – a geological ensemble for reservoir simulation," *Geoscience Data Journal*, vol. 1, no. 2, pp. 192–195, 2014.
- [9] A. F. Rasmussen, T. H. Sandve, K. Bao, A. Lauser, J. Hove, B. Skaflestad, R. Klöfkorn, M. Blatt, A. B. Rustad, O. Sævareid, K.-A. Lie, and A. Thune, "The Open Porous Media Flow reservoir simulator," *Computers & Mathematics with Applications*, vol. 81, pp. 159–185, 2021.

- [10] J. D. Jansen, “The Egg model – data files,” 4TU.ResearchData, TU Delft, 2013.
- [11] D. R. Brouwer and J. D. Jansen, “Dynamic optimization of waterflooding with smart wells using optimal control theory,” *SPE Journal*, vol. 9, no. 4, pp. 391–402, 2004.
- [12] A. A. Yousef, P. H. Gentil, J. L. Jensen, and L. W. Lake, “A capacitance model to infer interwell connectivity from production and injection rate fluctuations,” *SPE Reservoir Evaluation & Engineering*, vol. 9, no. 6, pp. 630–646, 2006.
- [13] M. Sayarpour, E. Zuluaga, C. S. Kabir, and L. W. Lake, “The use of capacitance–resistance models for rapid estimation of waterflood performance and optimization,” *Journal of Petroleum Science and Engineering*, vol. 69, pp. 227–238, 2009.
- [14] R. W. Holanda, E. Gildin, J. L. Jensen, L. W. Lake, and C. S. Kabir, “A state-of-the-art literature review on capacitance resistance models for reservoir characterization and performance forecasting,” *Energies*, vol. 11, no. 12, p. 3368, 2018.
- [15] A. Mamghaderi, B. Aminshahidy, and H. Bazargan, “Prediction of waterflood performance using a modified capacitance-resistance model: A proxy with a time-correlated model error,” *Journal of Petroleum Science and Engineering*, vol. 198, p. 108152, 2021.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [18] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable reinforcement learning implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [19] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” arXiv:2407.17032, 2024.
- [20] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa, “Safe exploration in continuous action spaces,” arXiv:1801.08757, 2018.
- [21] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.
- [22] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proc. 34th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, 2017, pp. 22–31.
- [23] Chen *et al.*, “Worst-case soft actor-critic-based safe reinforcement learning method for nonlinear constrained waterflood reservoir production optimization,” *SPE Journal*, vol. 30, no. 12, pp. 7745–7766, 2025.
- [24] J. Lei, M. G’Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman, “Distribution-free predictive inference for regression,” *Journal of the American Statistical Association*, vol. 113, no. 523, pp. 1094–1111, 2018.

- [25] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” arXiv:2107.07511, 2021.
- [26] I. Gibbs and E. Candès, “Adaptive conformal inference under distribution shift,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 1660–1672.
- [27] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [28] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, “Deep reinforcement learning at the edge of the statistical precipice,” in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 29 304–29 320.